

P0292R1: constexpr if: A slightly different syntax

Introduction

The paper P0128R1 "constexpr if" by Ville Voutilainen proposes a very useful facility to write compact template code that is instantiated depending on a compile-time condition. The motivation and rationale is sound and not repeated here.

This paper proposes a slightly different syntax, namely

```
if constexpr(cond)
    statement1;
else
    statement2;
```

The differences vs. P0128R1 are:

- `constexpr` is placed before the condition, indicating that the condition is evaluated as a compile-time constant.
- `constexpr` is not repeated before or after the `else`; an `else` binds to the nearest preceding if-statement as before (regardless of `constexpr`)
- a `constexpr if` is valid in a non-template and suppresses requirements for entity definitions in the non-taken branch
- `return` statements in a non-taken branch are ignored for return type deduction

This syntax avoids seemingly disingenous repetitions of `constexpr` in `if ... else if ...` chains:

```
// P0128R1
constexpr if (cond)
    statement1;
constexpr else constexpr if (cond)
    statement2;
constexpr else constexpr if (cond)
    statement3;
constexpr else
    statement4;
```

Compare with the syntax proposed in this paper:

```
if constexpr (cond)
    statement1;
else if constexpr (cond)
    statement2;
else if constexpr (cond)
    statement3;
else
    statement4;
```

The proposed syntax was approved by EWG during the Jacksonville (2016-03) meeting of WG21.

The wording below incorporates initial feedback from CWG.

Not proposed

Disarming `static_assert` declarations in the non-taken branch of a `constexpr if` is not proposed.

```
void f() {
    if constexpr (false)
        static_assert(false);    // ill-formed
}
```

```
template<class T>
void g() {
    if constexpr (false)
        static_assert(false);    // ill-formed; no diagnostic required for template definition
}
```

Changes compared to P0292R0

- slight wording improvements (see [blue](#) text)

Wording

The wording is based on the wording presented in P0128R1 with modifications for the changes explained above.

Change in 3.2 [basic.def.odr] paragraph 4:

Every program shall contain exactly one definition of every non-inline function or variable that is odr-used in that program **outside of a discarded statement (6.4.1 [stmt.if])**; no diagnostic required. The definition can appear explicitly in the program, it can be found in the standard or a user-defined library, or (when appropriate) it is implicitly defined (see 12.1, 12.4 and 12.8). An inline function shall be defined in every translation unit in which it is odr-used **outside of a discarded statement**.

Change in 6.4 [stmt.select] paragraph 1:

```
selection-statement:
    if constexpropt ( condition ) statement
    if constexpropt ( condition ) statement else statement
    switch ( condition ) statement
```

See 8.3 [dcl.meaning] for the optional *attribute-specifier-seq* in a condition. In Clause 6, the term substatement refers to the contained statement or statements that appear in the syntax notation. ...

Add a new paragraph after 6.4.1 [stmt.if] paragraph 1:

If the parenthesized *condition* is prefixed with `constexpr`, the condition shall be a contextually converted constant expression of type `bool` (5.20 [expr.const]); this form is called a *constexpr if* statement. If the value of the converted condition is false, the first substatement is a *discarded statement*, otherwise the second substatement, if present, is a discarded statement. A `case` or `default` label appearing within such an if statement shall be associated with a `switch` statement (6.4.2 [stmt.switch]) within the same if statement. A label (6.1 [stmt.label]) declared in a discarded statement shall not be referred to by a statement (6.6.4 [stmt.goto]) outside of a discarded statement. When a `constexpr if` statement appears in a templated entity, during an instantiation of the enclosing template or generic lambda, a discarded statement is not instantiated. [Example:

```
template<typename T, typename ... Rest> void g(T&& p, Rest&& ...rs) {
    // ... handle p
    if constexpr (sizeof...(rs) > 0)
        g(rs...); // never instantiated with an empty argument list.
}

extern int x; // no definition of x required
int f() {
    if constexpr (true)
        return 0;
    else if (x)
        return x;
    else
        return -x;
}
```

-- end example] [Note: Odr-uses (3.2 [basic.def.odr]) in a discarded statement do not require an entity to be defined. -- end note]

Change in 7.1.6.4 [dcl.spec.auto] paragraph 2:

... If the declared return type of the function contains a placeholder type, the return type of the function is deduced from **non-discarded** return statements, **if any**, in the body of the function, **if any** (6.4.1 [stmt.if]).

Change in 7.1.6.4 [dcl.spec.auto] paragraph 7:

When a variable declared using a placeholder type is initialized, or a **non-discarded** return statement occurs in a function declared with a return type that contains a placeholder type, the deduced return type or variable type is determined from the type of its initializer. ...

Change in 7.1.6.4 [dcl.spec.auto] paragraphs 9-11:

If a function with a declared return type that contains a placeholder type has multiple **non-discarded** return statements, the return type is deduced for each return statement. If the type deduced is not the same in each deduction, the program is ill-formed.

If a function with a declared return type that uses a placeholder type has no **non-discarded** return statements, the return type is deduced as though from a return statement with no operand at the closing brace of the function body. [Example: ...]

Once a **non-discarded** return statement has been seen in a function, however, the return type deduced from that statement can be used in the rest of the function, including in other return statements. ...

Change in 14.5 [temp.decls] paragraph 2 as follows:

For purposes of name lookup and instantiation, default arguments and *exception-specifications* of function templates and default arguments and *exception-specifications* of member functions of class templates are considered definitions; each default argument or *exception-specification* is a separate definition which is unrelated to the function template definition or to any other default arguments or *exception-specifications*. **For the purpose of instantiation, the substatements of a constexpr if statement (6.4.1 [stmt.if]) are considered definitions.**

Change in 14.6 [temp.dep] paragraph 8:

... If no valid specialization can be generated for a template **or a substatement of a constexpr if statement (6.4.1 [stmt.if]) within a template**, and ~~that~~ **the** template is not instantiated, the template is ill-formed, no diagnostic required. ...

Change 14.7.1 [temp.inst] paragraph 11 as follows:

An implementation shall not implicitly instantiate a function template, a variable template, a member template, a non-virtual member function, a member class, ~~or a static data member of a class template,~~ **or a substatement of a constexpr if statement (6.4.1 [stmt.if]) that does not require instantiation**, **unless such instantiation is required.**

Acknowledgements

Thanks to Walter Brown for suggestions for wording improvement.